

Elementary Data Structures: Stacks and Queues

CMPT 211: Based on CLRS Algorithms (Chapter 10)

Bernal Manzanilla Saavedra

Concordia University of Edmonton

Winter 2026

Introduction to Elementary Data Structures

In computer science, we frequently need to maintain **dynamic sets** of data—collections of items that grow, shrink, and change over time.

According to Chapter 10 of *Introduction to Algorithms* (CLRS), we can represent complex dynamic sets using simple, contiguous memory arrays.

The Core Concept: Bounded Operations

- We use an array to store the data.
- However, we do not allow the user to arbitrarily insert or delete elements anywhere in the array.
- Instead, we strictly control *where* modifications can happen by maintaining internal **pointers** (indices).

The Stack Data Structure

A **Stack** is a dynamic set that implements a **Last-In, First-Out (LIFO)** policy.

Think of a physical stack of plates in a cafeteria. You can only place a new plate on the very top, and you can only remove a plate from the very top.

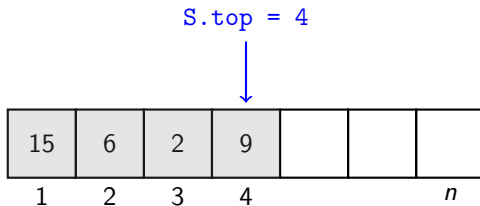
CLRS Operations:

- **PUSH(S, x)**: Inserts the element x into stack S .
- **POP(S)**: Removes and returns the most recently inserted element.
- **STACK-EMPTY(S)**: Returns true if the stack contains no elements.

In an array-based implementation, we manage this using a single integer attribute: $S.top$, which points to the index of the most recently inserted element.

Visualizing the Stack (CLRS Figure 10.1)

Let's visualize an array $S[1 \dots n]$. The active elements of the stack reside in positions $S[1 \dots S.top]$.



- If we call $PUSH(S, 17)$, we increment $S.top$ to 5, and write 17 into $S[5]$.
- If we call $POP(S)$, we return the value at $S[S.top]$ (which is 9), and then decrement $S.top$ back to 3.

Translating CLRS Pseudocode to Python

When `S.top` equals 0, the stack is empty. If we attempt to pop an empty stack, we trigger an error called **Underflow**.

```
class Stack:
    def __init__(self, capacity):
        self.S = [None] * capacity # The underlying array
        self.top = -1 # Using 0-indexed arrays in Python

    def stack_empty(self):
        return self.top == -1

    def push(self, x):
        # We omit overflow checking here for brevity
        self.top = self.top + 1
        self.S[self.top] = x

    def pop(self):
        if self.stack_empty():
            raise Exception("Stack Underflow: Cannot pop an empty stack.")
        else:
            self.top = self.top - 1
            return self.S[self.top + 1] # Return the element we just stepped away
```

The Queue Data Structure

A **Queue** implements a **First-In, First-Out (FIFO)** policy.

Think of a line of customers waiting at a checkout. The first person to enter the line is the first person to be served.

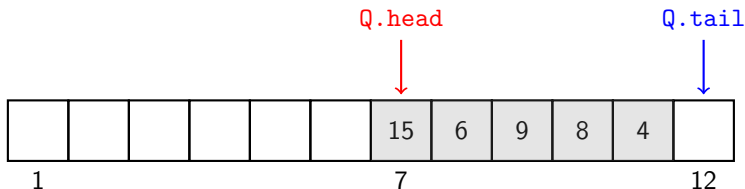
CLRS Operations:

- **ENQUEUE(Q, x)**: Inserts element x at the *tail* of the queue.
- **DEQUEUE(Q)**: Removes and returns the element at the *head* of the queue.

To implement this efficiently in an array, we must maintain TWO pointers: $Q.head$ (where data leaves) and $Q.tail$ (where new data is inserted).

Visualizing the Queue (CLRS Figure 10.2)

The elements in the queue reside in continuous locations from $Q.head$ to $Q.tail - 1$.



- **Enqueue($Q, 17$):** We write 17 into index 12, and increment $Q.tail$ to 13.
- **Dequeue(Q):** We read the value 15 at index 7, and increment $Q.head$ to 8. The data literally "crawls" to the right across the array memory!

The Wrap-Around (Circular Array)

If the data continually crawls to the right, what happens when `Q.tail` reaches the very end of the array at index n ?

If there is empty space at the beginning of the array (because `Q.head` has also moved right), we wrap `Q.tail` back around to index 1! This treats the linear array memory as if it were a **circular** ring.

Mathematical Implementation: We achieve this wrap-around effortlessly using the **modulo operator** (%). If our array length is N , then setting `Q.tail = (Q.tail + 1) % N` ensures the pointer instantly resets to 0 when it overshoots the end of the array.

Summary

Stacks (LIFO) and Queues (FIFO) are logical abstractions.

Under the hood, they are simply static memory arrays manipulated by strict pointer boundaries (`top`, `head`, `tail`) preventing unauthorized data access.